

用户数据采集与关联分析 结课作业

W O R K R E P O R T

信管2302张雨暄 202321054035

PART 01

课程导言与分词



1.

```
1 import pynlpir
2
3 pynlpir.open()
4 content = open('F:/Cous/3-1/DA/homework/selfwork/01seg/huangxuhua.txt','r',encoding = 'utf-8').read()
5 pynlpir.nlpir.ImportUserDict(b"F:/Cous/3-1/DA/homework/selfwork/01seg/dic.txt")
6 sw=open('F:/Cous/3-1/DA/homework/selfwork/01seg/stopwords.txt','r',encoding = 'utf-8').read()
7
8 #分词
9 words = pynlpir.segment(content,pos_tagging=False, pos_names='parent', pos_english=False)
10 for word in words:
11     if word not in sw and len(word)>1 and True not in [c.isdigit() for c in word]:
12         print(word,end='')
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

整理/著录/大部分/资料/数字化/黄旭华/学术/资料/清单/黄旭华/学术/资料/实物/清单/黄旭华/学术/资料/原件/清单/利用/基础/采集/小组/资料/系统/组织/研究/编制/黄旭华/年表/年表/脉络/编辑/黄旭华/学术/资料/长编/采集/黄旭华/学术/资料/井然有序/利于/将来/科学/研究/深度/开发/采集/小组/工作/科学家/学术/成长/资料/采集/工程/撰写/黄旭华/院士/传记/传记/写作/采集/工程/主旨/意义/分析/黄旭华/成长/时代/背景/成长/轨迹/采集/小组/传记/写作/思路/第一/完整/有序/传主/童年/人生/历程/重点/还原/传主/曲折/求学/地下党/生涯/核潜艇/研制/核心/事件/历史/进程/第三/力图/理清/传主/人生/转折/学术/成长/科学/成就/思想/背景/因果/关联/历史/经纬/第四/准确/铺陈/传主/科学/成就/客观/评价/科学/地位/历史/贡献/黄旭华/传记/章节/时间/事件/布局/章节/大小/主题/渐次/叙述/传记/事实/准确/客观/公允/描述/流畅/清晰/争议/客观/陈述/结论/评价/原则/展示/黄旭华/家庭/背景/童年/生活/重点/介绍/黄旭华/父母亲/道德/修养/文化/背景/职业/操守/性格/禀赋/心/此/追索/黄旭华/精神/品德/媒体/传播/生日/求证/还原/人生/真实/起点/叙述/黄旭华/曲折/求学/过程/人生/目标/黄旭华/求学/生涯/戏剧/曲折/期间/充满/艰辛/求学/描述/展示/黄旭华/知识/收获/优秀/教师/影响/动荡/时局/心灵/洗礼/最终/揭示/人生/目标/原因/国立交通大学/学习/生活/革命/斗争/黄旭华/人生/第一/高潮/黄旭华/夯实/学术/成长/起点/学生/坚强/革命者/华丽/蜕变/第三/笔者/力图/铺叙/黄旭华/专业/学习/情况/介绍/交通大学/一心/代表/船舶/制造/先驱/黄旭华/专业/思想/培育/定位/学术/成长/时间/渐次/展开/黄旭华/参加/山茶/学生/社团/学生/运动/画卷/还原/波谲云诡/惊心动魄/革命/斗争/过程/尝试/追溯/参加/地下党/思想/变化/发轫/献身/核潜艇/事业/精神/基础/第四/介绍/黄旭华/交通大学/毕业/直至/参加/工程/工作/经历/看似/过渡性/章节/描述/黄旭华/思想/观念/成熟/专业/技能/提升/磨炼/过程/暗示/黄旭华/科学/成就/思想/专业/技术/渊源/学术/成长/规律/轨迹/过渡/思想/成熟/专业/技能/蓄积/环节/笔者/命名/十年磨一剑/工程/黄旭华/人生/转折点/时代/给予/机遇/第五/介绍/我国/核潜艇/研制/时代/背景/缘起/描述/黄旭华/工程/过程/介绍/核潜艇/研制/开拓性/工作/叙述/核潜艇/工程/下马/分析/下马/原因/第六/七两/我国/核潜艇/研制/历史/坐标/逐一/介绍/核潜艇/研制/历史/节点/黄旭华/承担/时间/黄旭华/学术/成长/成熟期/收获/一生/学术/思想/专业/成就/阶段/逐一/回顾/黄旭一/介绍/核潜艇/研制/历史/节点/黄旭华/承担/时间/黄旭华/学术/成长/成熟期/收获/一生/学术/思想/专业/成就/阶段/逐一/回顾/黄旭华/核潜艇/研制/亲历/事件/再现/学术/成熟/脉络/分析/创新/思想/技术/突破/科学/精神/第八/黄旭华/科学/贡献/人生/追求/展示/退居二线/核潜艇/发展/忠贞/情怀/重点/黄旭/华为/代表/第一/核潜艇/铸造/核潜艇/精神/本质/特征/剖析/第九/黄旭华/和谐/家庭/幸福/生活/家人/事业/坚定/支持/理解/本章/夫人/李世英/重点/介绍/品德/品质/品位/李世英/塑造/近乎/完美/黄旭华/人生/提升/人生/价值/诠释/知识/女性/成功/

2.

```
[2]: print("Hello World.")
```

Hello World.

```
[3]: print("Hello Zhang Yuxuan!")
```

Hello Zhang Yuxuan!

3.

```
In [1]: import jieba
```

```
In [2]: seg_list=jieba.cut("曾经有一份真诚的爱情摆在我的面前，我没有珍惜，等到失去的时候才追悔莫及，人世间最痛苦的事情莫过于此。如果上天能够给我一")
```

```
In [3]: print(' '.join(seg_list))
```

```
Building prefix dict from the default dictionary ...
Dumping model to file cache C:\Users\ChErRy\AppData\Local\Temp\jieba.cache
Loading model cost 0.655 seconds.
Prefix dict has been built successfully.
```

曾经/有/一份/真诚/的/爱情/摆在我/的/面前/，/我/没有/珍惜/，/等到/失去/的/时候/才/追悔莫及/，/人世间/最/痛苦/的/事情/莫过于此/。/如果/上天/能够/给/我/一个/重新/来/过/的/机会/，/我会/对/那个/女孩子/说/三个/字/：/ ‘/我爱你/’ /。/如果/非要/给/这份/爱/加上/一个/期限/，/我/希望/是/，/一万年

```
In [4]: seg_list=jieba.cut("LSTM (Long Short-Term Memory) 是长短期记忆网络，是一种时间递归神经网络，适合于处理和预测时间序列中间隔和延迟相对较长的")
```

```
In [5]: print(' '.join(seg_list))
```

LSTM/ (/Long/ /Short/-/Term/ /Memory/) /是/长短期/记忆/网络/，/是/一种/时间/递归/神经网络/，/适合/于/处理/和/预测/时间/序列/中/间隔/和/延迟/相对/较长/的/重要/事件/。

```
In [12]: jieba.load_userdict('dict.txt')
```

```
In [13]: seg_list=jieba.cut("LSTM (Long Short-Term Memory) 是长短期记忆网络，是一种时间递归神经网络，适合于处理和预测时间序列中间隔和延迟相对较长的")
```

```
In [14]: print(' '.join(seg_list))
```

LSTM/ (/Long/ /Short/-/Term/ /Memory/) /是/长短期记忆网络/，/是/一种/时间递归神经网络/，/适合/于/处理/和/预测/时间序列/中/间隔/和/延迟/相对/较长/的/重要/事件/。

```
In [16]: stopwords = [line.strip() for line in open('stop_words.txt', 'r', encoding='utf-8').readlines()]
```

```
In [17]: seg_list=jieba.cut("曾经有一份真诚的爱情摆在我的面前，我没有珍惜，等到失去的时候才追悔莫及，人世间最痛苦的事情莫过于此。如果上天能够给我一")
```

```
In [18]: final=''
```

```
In [19]: for seg in seg_list:
          if seg not in stopwords:
              final+=seg+'/'
```

```
In [20]: print(final)
```

曾经/有/一份/真诚/爱情/摆在/我/面前/我/没有/珍惜/等到/失去/时候/才/追悔莫及/人世间/最/痛苦/事情/莫过于此/如果/上天/能够/给/我/一个/重新/来/过/机会/我会/对/那个/女孩子/说/三个/字/我爱你/如果/非要/给/这份/爱/加上/一个/期限/我/希望/一万年/

```
# stopwords = [line.strip() for line in open('stop_words.txt', 'r', encoding='utf-8').readlines()]
```

```
stopwords = open('stop_words.txt', 'r', encoding='utf-8').read()
stopwords = stopwords.split('\n')
```

```
stopwords
```

```
['的', '了', '是', '啊', '、', '，', '。', '！', '停用']
```

```
seg_list_huang = jieba.cut('黄旭华，1926年3月12日出生于广东省汕尾市，原籍广东省揭阳市。1949年毕业于上海交通大学。历任北京海军核潜艇研究室副总工程师、中船重工集团公
```

```
final = ''
```

```
for seg in seg_list_huang:
    if seg not in stopwords:
        final+= seg+'/'
```

```
print(final)
```

黄旭华/1926/年/3/月/12/日出/生于/广东省/汕尾市/原籍/广东省/揭阳市/1949/年/毕业/于/上海交通大学/历任/北京/海军/核潜艇/研究室/副/总工程师/中船重工集团公司/核潜艇/总体/研究/设计所/研究员/名誉/所长/1994/年/当选/为/中国工程院院士/

```
print("\n所有词汇词频统计（前20个）：")
for word, count in word_counts.most_common(20):
    print(f'{word}: {count}次')
```

所有词汇词频统计（前20个）：

'，': 13次
' '： 9次
'管理': 5次
'、': 5次
'与': 4次
'“': 3次
'企业': 3次
'”': 3次
'体系': 3次
'智能化': 3次
'经营': 3次
'打造': 3次
'能力': 3次
'供应链': 3次
'建设': 2次
'通过': 2次
'自动化': 2次
'数字化': 2次
'升级': 2次
'生产': 2次

4.

《基于学术论文全文的研究方法句自动抽取研究》聚焦科技文献中研究方法句的自动抽取问题，为科研工作者快速获取合适研究方法提供解决方案。

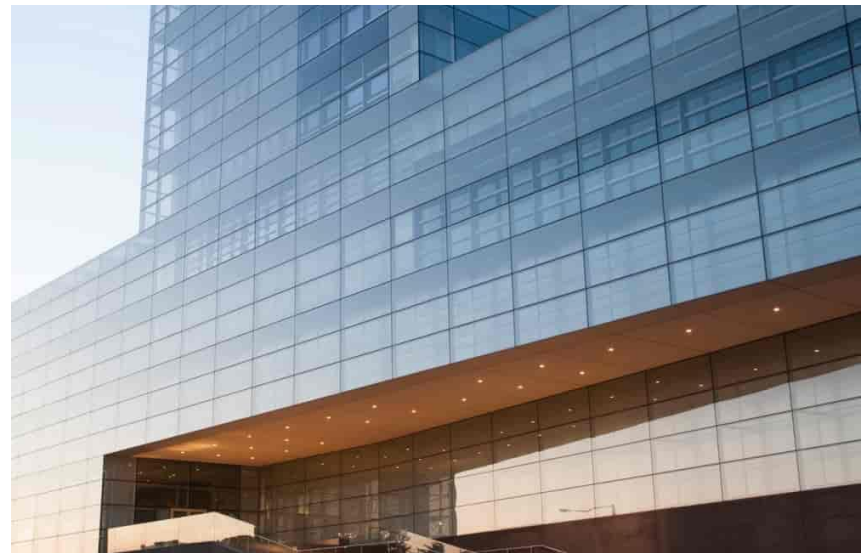
论文创新点突出，首次将研究方法句分为论文使用方法句和论文引用方法句两类，填补了现有研究对其定义和分类的空白。研究采用多种深度学习模型，在词向量表示层对比 BERT 和 word2vec，特征选择层选用卷积神经网络、双向长短时记忆网络等三种网络，还尝试了单层次和两层次两种训练结构。

实验以《情报学报》2009-2018 年的 1170 篇论文为数据基础，构建人工标注数据集并划分训练集、验证集和测试集。结果显示，基于 BERT 的单层次结构双向长短时记忆网络模型性能最优，在多项评测指标上表现突出。

通过对抽取结果的分析，发现《情报学报》近 10 年研究方法句分布呈现特定趋势，反映出情报学学科逐渐重视理论发展与学科理论体系建设。该研究为学术文献研究方法挖掘提供了有效技术路径，后续可通过半监督学习解决标注数据不足问题，并扩大期刊数据范围。

PART 02

词频统计



1.

主题分布

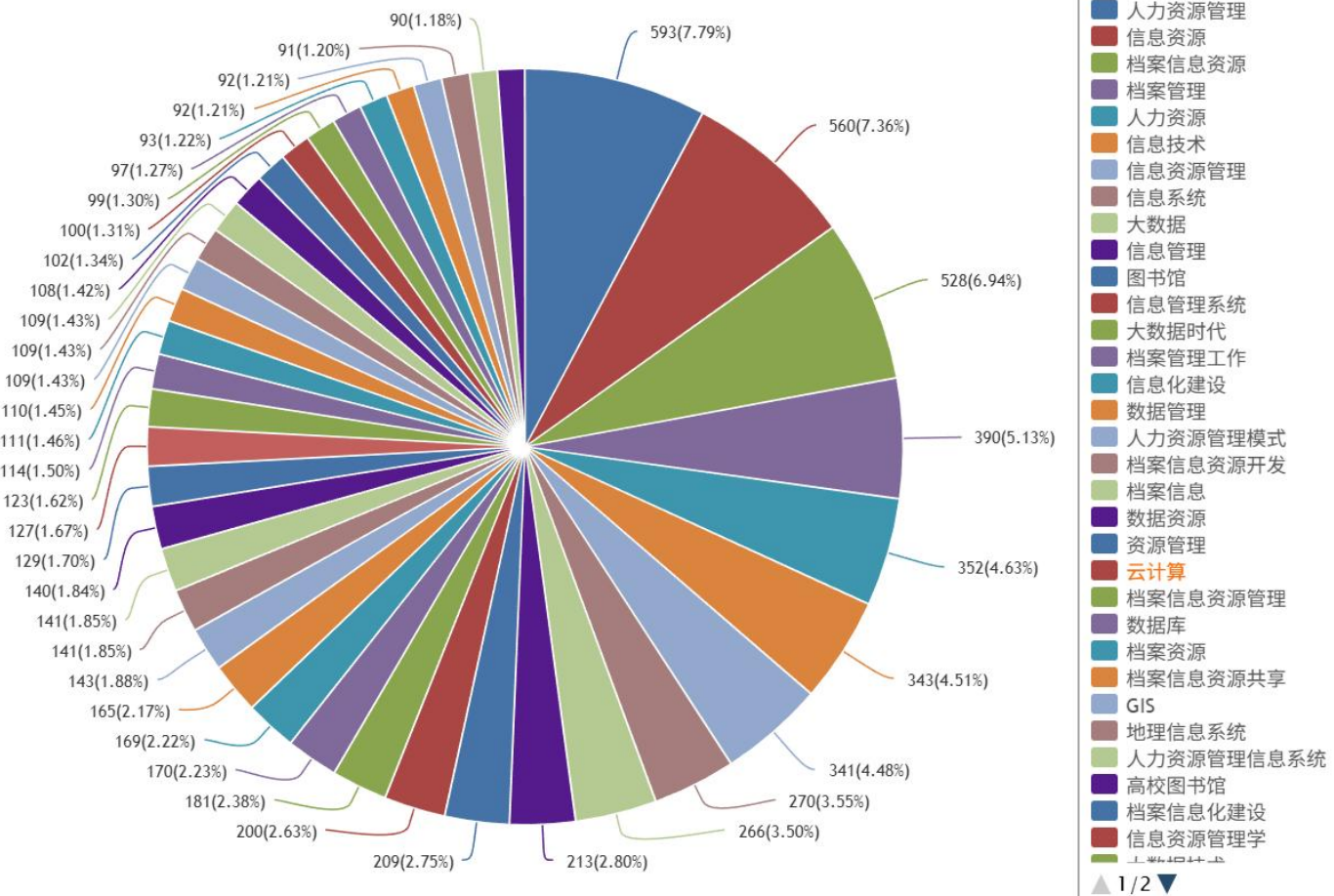
数据库: 期刊

检索条件: (主题: 信息资源...)

分布项: 主题

显示数量:

10 20 30 40 50 60 70 80 90 100



2.

```
[1]: import jieba

[2]: article = open('科学家博物馆-黄旭华传记序言.txt','r',encoding = 'utf-8').read() # 打开并读取三国前10回 #出现乱码提示, 就把ANSI改成utf-8

[3]: dele = '。','!','?','的','''','(',')','(',')','(',')','(',')','(',')' # 手动设计一些停用词和符号

[7]: jieba.add_word('国立交通大学') # 加入字典中没有的新词

[8]: words = list(jieba.cut(article)) # 给巴分词出来的词汇

[9]: words
```

```
[11]: articleSet = set(words)-dele
```

```
[12]: articleSet
```

```
[12]: {'\n',
      '09',
      '1020',
      '1924',
      '1949',
      '1958',
      '1978',
      '1982',
      '1985',
      '1986',
      '1989',
      '1996',
      '2',
      '2013',
      '24',
      '29',
```

```
[1]: !pip install jieba
```

```
Requirement already satisfied: jieba in /Users/wuzhixiang/anaconda3/lib/python3.11/site-packages (0.42.1)
```

```
[1]: import jieba
```

```
[2]: article = open('sanguo_10.txt', 'r', encoding = 'utf-8').read() # ansi/打开并读取三国前10回 #出现乱码提示, 就把ANSI改成utf-8
```

```
[3]: dele = {'。', '!', '?', '的', '"', "'", '(', ')', ' ', ' ', '》', '《', ' ', ' '} # 手动设计一些停用词和符号
```

```
[4]: jieba.add_word('羅貫中') # 加入字典中没有的新词
```

```
Building prefix dict from the default dictionary ...
Loading model from cache /var/folders/lr/4bl1bljn0fx_lrktr8wvmvg9r0000gn/T/jieba.cache
Loading model cost 0.345 seconds.
Prefix dict has been built successfully.
```

```
[5]: words = list(jieba.cut(article)) #结巴分词出来的词汇
```

```
[6]: words
```

```
[31]: weapons_origin = data_weapon.split('\n')

[32]: print(weapons_origin[:40])

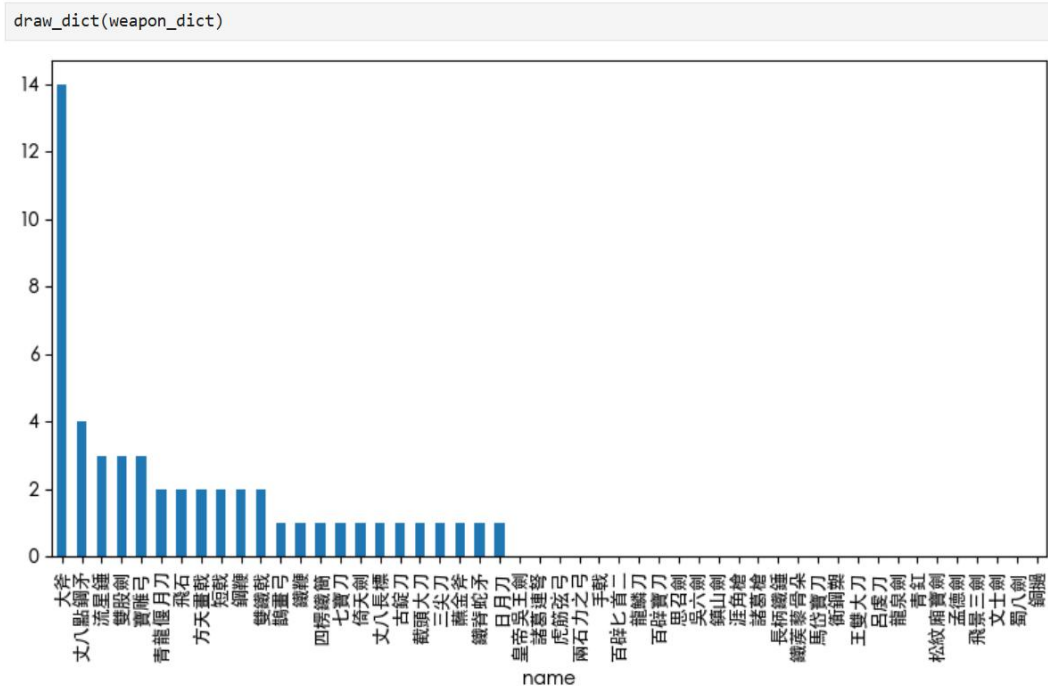
['青龍偃月刀', '', '丈八點鋼矛', '', '鐵脊蛇矛', '', '涯角槍', '', '諸葛槍', '', '方天畫戟', '', '長柄鐵錘', '', '鐵蒺藜骨朵', '', '大斧', '', '蘸金斧', '', '三尖刀', '', '截頭大刀', '', '馬岱寶刀', '', '古錠刀', '', '衝鋼槊', '', '丈八長標', '', '王雙大刀', '', '呂虔刀', '', '龍泉劍', '', '倚天劍', '']

[33]: weapons = []

[34]: for weapon in weapons_origin:
        if weapon != '':
            weapons.append(weapon)

[35]: weapons[:500]

['青龍偃月刀',
'丈八點鋼矛',
'鐵脊蛇矛',
'涯角槍',
'諸葛槍',
'方天畫戟',
'長柄鐵錘',
'鐵蒺藜骨朵',
'大斧',
'蘸金斧',
'三尖刀',
'截頭大刀',
'馬岱寶刀',
'古錠刀',
'衝鋼槊',
'丈八長標',
'王雙大刀',
'呂虔刀',
'龍泉劍',
'倚天劍',
```



3.

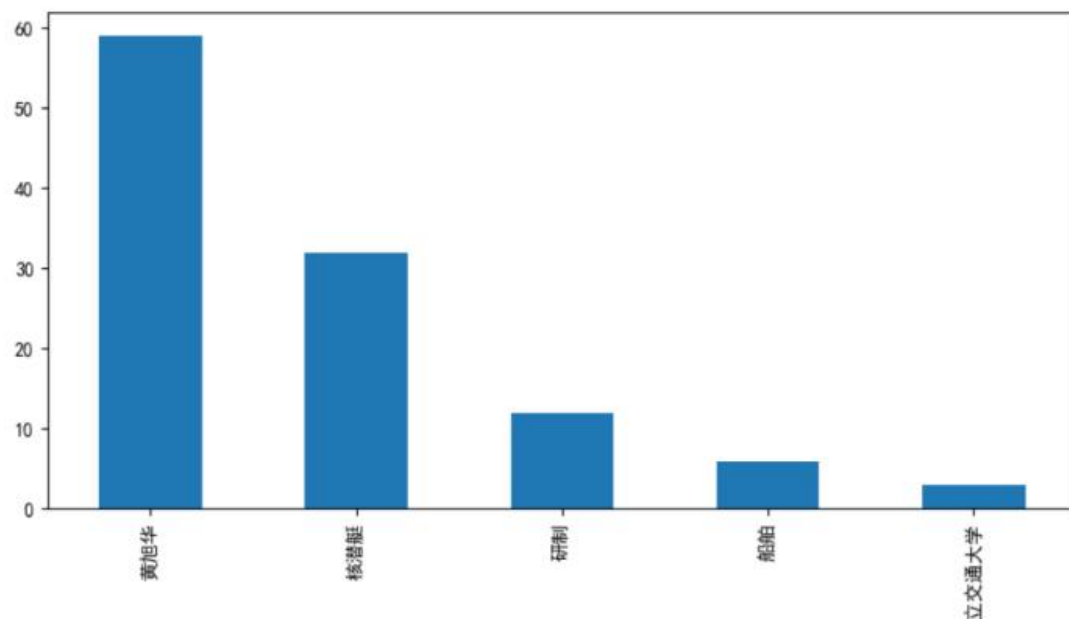
```
In [17]: # 输出词频的前N个
for i in range(100):
    print(articlelist[i])
```

```
('黄旭华', 55)
('核潜艇', 32)
('采集', 29)
('学术', 22)
('资料', 21)
('工作', 17)
('成长', 15)
('小组', 14)
('专业', 13)
('院士', 13)
('进行', 13)
('技术', 12)
('我国', 12)
('研制', 12)
('工程', 11)
('访谈', 10)
('科学', 8)
('介绍', 8)
('第一代', 8)
('主要', 8)
```

```
In [29]: for term in terms:
        terms_dict[term]=data_txt.count(term)
```

```
In [30]: terms_dict
```

```
Out[30]: {'黄旭华': 59, '核潜艇': 32, '国立交通大学': 3, '船舶': 6, '研制': 12}
```



4.

该研究借助谷歌图书和谷歌学术的海量数字化资源，聚焦物理学家的科学声誉，通过量化分析探索其跨世纪影响力演变。研究以牛顿和爱因斯坦为核心案例，结合 234 位顶尖物理学家的数据，揭示了科学声誉的长期传承规律与影响因素。

研究创新采用谷歌图书（3600 万册）和谷歌学术（9100 万篇文献）的多语言语料库，以姓名提及频率为核心指标，辅以共现分析等方法。核心发现包括：伟大科学家的声誉并未随逝世消亡，牛顿和爱因斯坦的影响力跨越数世纪，且 20 世纪 80 年代后关注度显著回升；存在“群体内声誉偏好”，牛顿在英式英语语境中更受推崇，爱因斯坦则在美式英语、德语等语境中更具影响力；两人的声誉分别与万有引力定律、相对论等核心成就高度相关，但也受其哲学思想、生平轶事等因素加持。

此外，研究列出 21 世纪最具声誉的 20 位物理学家，爱因斯坦、普朗克、牛顿位居前三，印证了早期科学家的持久影响力。该研究还指出，谷歌图书和 Ngram Viewer 可作为替代计量工具，弥补传统引文计量在长期社会影响力评估中的不足。

PART 03

词云与可视化



1.



《罗杰疑案》地点变化词云图

2.

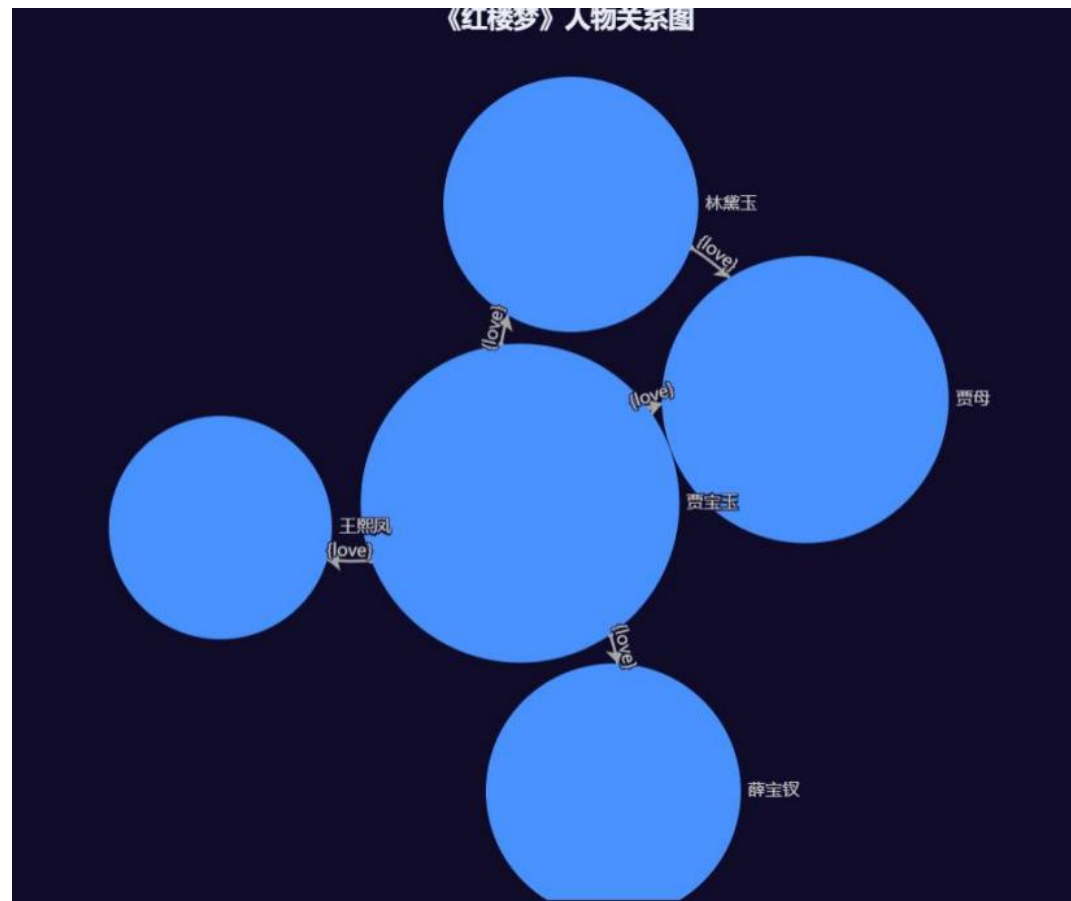
```

var nodes = [
  { name: '贾宝玉', symbolSize: 50 },
  { name: '林黛玉', symbolSize: 40 },
  { name: '薛宝钗', symbolSize: 40 },
  { name: '王熙凤', symbolSize: 35 },
  { name: '贾母', symbolSize: 45 },
];

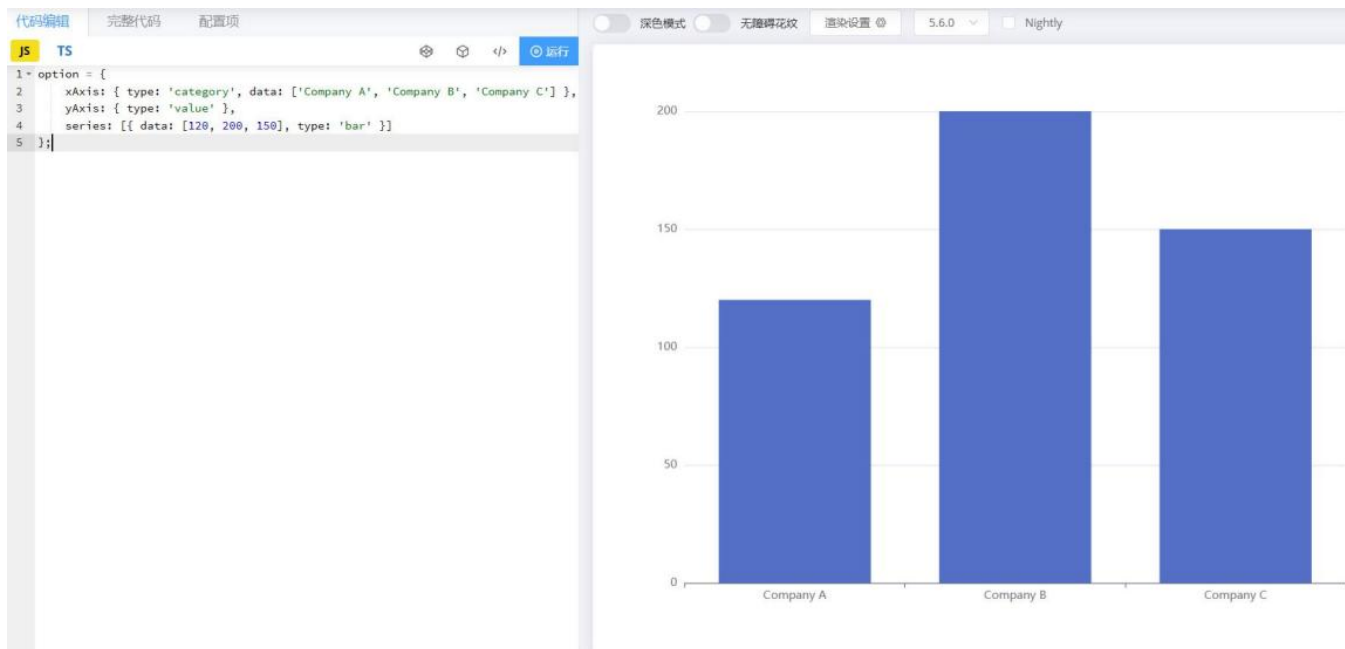
var edges = [
  { source: '贾宝玉', target: '林黛玉' },
  { source: '贾宝玉', target: '薛宝钗' },
  { source: '贾宝玉', target: '王熙凤' },
  { source: '贾宝玉', target: '贾母' },
  { source: '林黛玉', target: '贾母' },
];

option = {
  title: {
    text: '《红楼梦》人物关系图',
    left: 'center'
  },
  tooltip: {},
  series: [{
    type: 'graph',
    layout: 'force', // 使用力导向布局
    edgeLength: 2500,
    symbolSize: 50, // 默认节点大小
    roam: true, // 允许缩放和平移
    label: {
      show: true,
      position: 'right'
    },
    edgeSymbol: ['circle', 'arrow'], // 边两端的符号
    edgeSymbolSize: [4, 10], // 边两端符号大小
    edgeLabel: {
      show: true,
      formatter: '{love}' // 显示边上的文本
    },
  ]
};

```

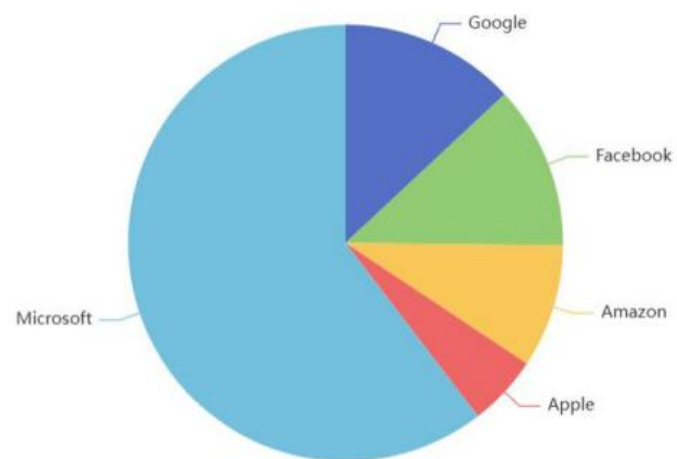


2.



```

option = {
  series: [
    {
      type: 'pie',
      radius: '50%',
      data: [
        { value: 335, name: 'Google' },
        { value: 310, name: 'Facebook' },
        { value: 234, name: 'Amazon' },
        { value: 135, name: 'Apple' },
        { value: 1548, name: 'Microsoft' }
      ]
    }
  ]
};
  
```



3.



4.

```
Populating the interactive namespace from numpy and matplotlib
(-0.5, 399.5, 199.5, -0.5)
```



```
[22]: # 重新生成词云
from wordcloud import WordCloud
wordcloud_cn = WordCloud(font_path="simsun.ttf").generate(final)
%pylab inline
import matplotlib.pyplot as plt
plt.imshow(wordcloud_cn, interpolation='bilinear')
plt.axis('off')

%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

[22]: (-0.5, 399.5, 199.5, -0.5)
```



```
[23]: big_pic = imread('huangxuhua.jpg')

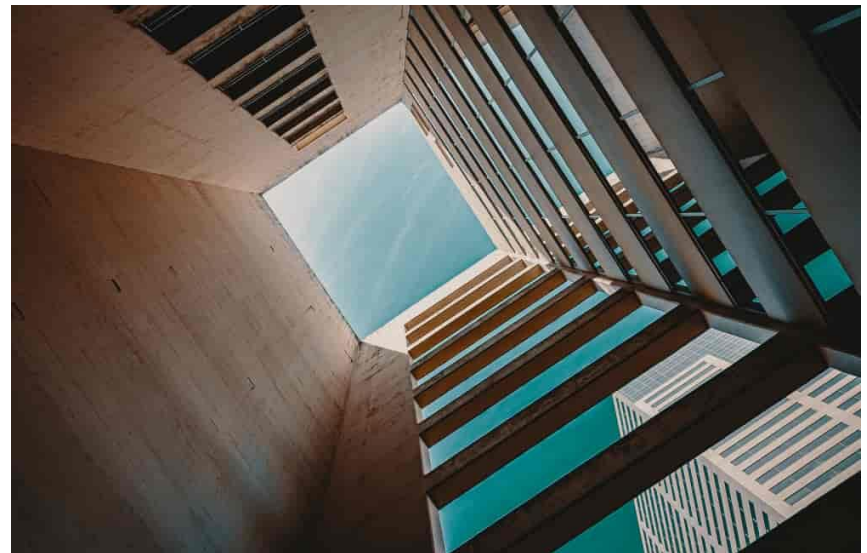
[24]: wordcloud_cn_x = WordCloud(mask=big_pic,width=500,height=300, background_color='white',scale=2.5, font_path="simsun.ttf").generate(final)
plt.imshow(wordcloud_cn_x)
plt.axis('off')

[24]: (-0.5, 1374.5, 1924.5, -0.5)
```

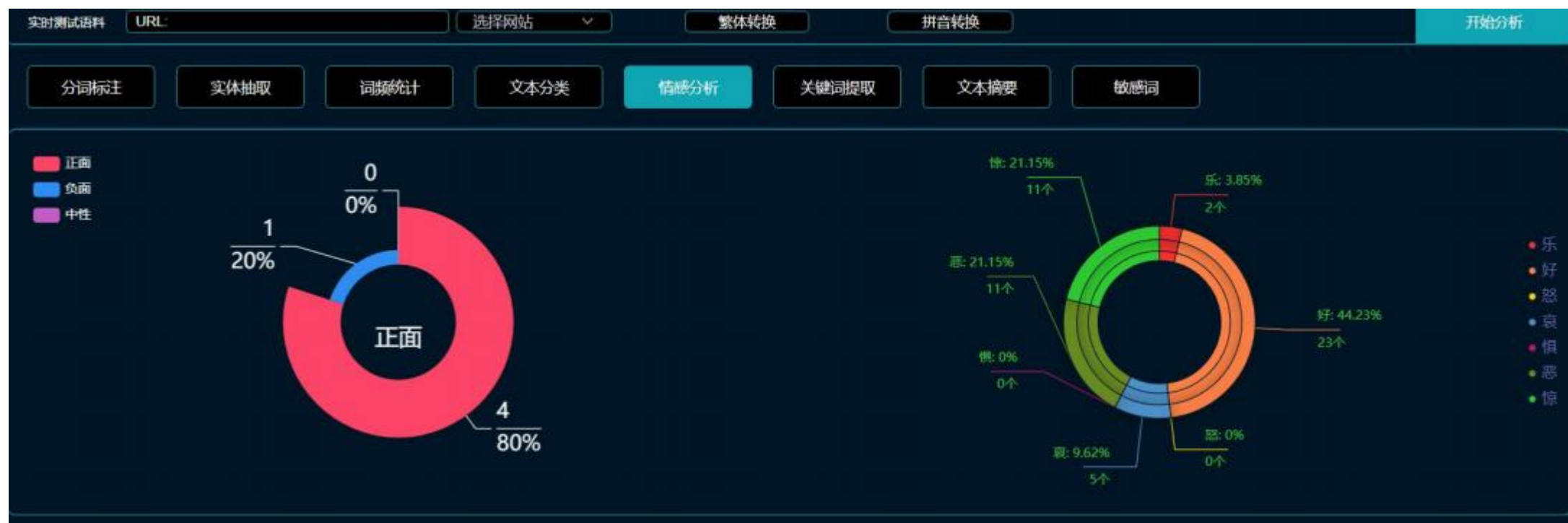


PART 04

情感分析



1.



2.

例句1

```
text_taobao_4 = "商品收到了，质量超出预期与描述完全一致。包装仔细，服务态度好，物流速度快。商品做工精致，面料柔软，颜色无偏差，穿着很舒适"
```

```
taobao_4 = SnowNLP(text_taobao_4)
```

```
taobao_4.sentiments
```

```
0.9989841874600173
```

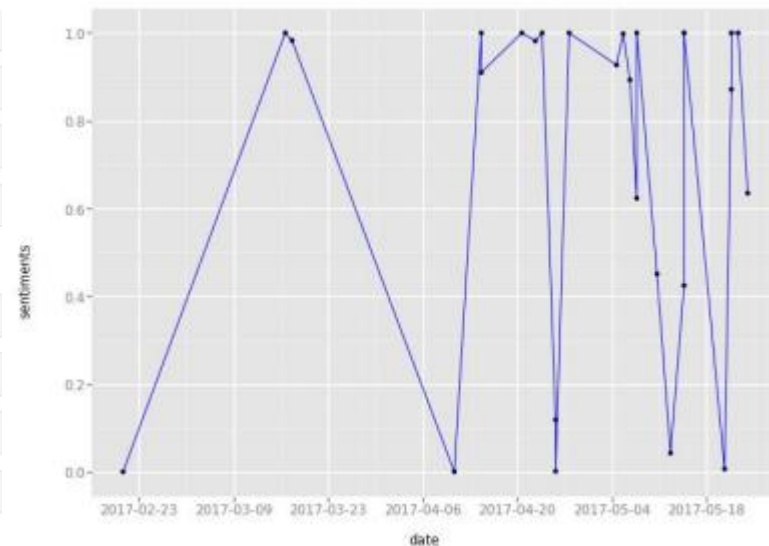
例句2

```
text_taobao_5 = "these wireless earbuds have been a game-changer for my daily routines"
```

```
taobao_5 = SnowNLP(text_taobao_5)
```

```
taobao_5.sentiments
```

```
0.2619295992013928
```



[23]: %pylab inline

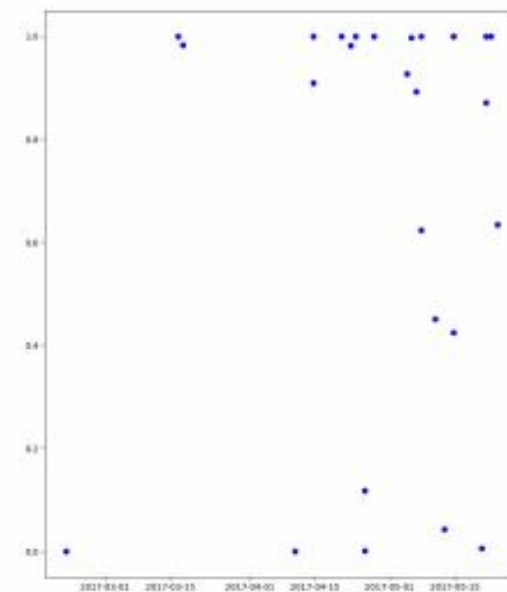
%pylab is deprecated, use %matplotlib inline and import the required libraries.
Populating the interactive namespace from numpy and matplotlib

C:\Users\I7136\anaconda3\Lib\site-packages\IPython\core\magics\pylab.py:166: UserWarning: pylab import has clobbered these variables: ['text']
%matplotlib prevents importing * from pylab and numpy
warn("pylab import has clobbered these variables: %s" % clobbered +

[24]: import matplotlib.pyplot as plt
#plt.scatter(df["date"],df["sentiments"],color='blue',figsize=(10,12))
plt.figure(figsize=(10, 12))
plt.scatter(df["date"],df["sentiments"],color='blue')

[24]: <matplotlib.collections.PathCollection at 0x209467586b0>

[25]: plt.savefig('timeline.png') # 要不要? 点一点?
plt.show()



jupyter sentiment analysis_4 大模型 朱自清 背影 Last Checkpoint: 上星期一-21:08 (autosaved)



Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted

Python 3 ○

- 3) 用大语言模型计算出，每一个情感词或者情感词所在句子的情感得分；
- 4) 然后绘制一个时间序列的可视化图，横坐标是句子的序号，纵坐标是情感得分，每一个散点是句子对应的情感词，情感词作为散点的标签，然后用虚线把这些情感词连起来。

```
In [2]: import re
```

```
with open("背影.txt", "r", encoding="utf-8") as f:
    text = f.read()

# 按照句号、感叹号进行分句（保留标点）
sentences = re.split('(?<=[。！])', text)
sentences = [s.strip() for s in sentences if s.strip()]
numbered_sentences = [(i + 1, s) for i, s in enumerate(sentences)]
# numbered_sentences = numbered_sentences[0:5] #大模型抽取很慢，提取文本的前n条句子

for num, sent in numbered_sentences:
    print(f"{num}: {sent}")
```

- 1: 我与父亲不相见已二年余了，我最不能忘记的是他的背影。
- 2: 那年冬天，祖母死了，父亲的差使也交卸了，正是祸不单行的日子。
- 3: 我从北京到徐州，打算跟着父亲奔丧回家。
- 4: 到徐州见着父亲，看见满院狼藉的东西，又想起祖母，不禁簌簌地流下眼泪。
- 5: 父亲说：“事已如此，不必难过，好在天无绝人之路！”
- 6: ”
- 回家变卖典质，父亲还了亏空；又借钱办了丧事。
- 7: 这些日子，家中光景很是惨淡，一半为了丧事，一半为了父亲赋闲。
- 8: 丧事完毕，父亲要到南京谋事，我也要回北京念书，我们便同行。
- 9: 到南京时，有朋友约去游逛，勾留了一日；第二日上午便须渡江到浦口，下午上车北去。
- 10: 父亲因为事忙，本已说定不送我，叫旅馆里一个熟识的茶房陪我同去。
- 11: 他再三嘱咐茶房，甚是仔细。
- 12: 但他终于不放心，怕茶房不妥帖；颇踌躇了一会。
- 13: 其实我那年已二十岁，北京已来往过两三次，是没有什么要紧的了。
- 14: 他踌躇了一会，终于决定还是自己送我去。
- 15: 我再三劝他不必去；他只说：“不要紧，他们去不好！”
- 16: ”
- 我们过了江，进了车站。
- 17: 我买票，他忙着照看行李。
- 18: 行李太多了，得向脚夫们付些小费才可过去。
- 19: 他便又忙着和他们讲价钱。
- 20: 我那时真是聪明过分，总觉他说话不大漂亮，非自己插嘴不可，但他终于讲定了价钱；就送我上车。
- 21: 他给我拣定了靠车门的一张椅子；我将他给我做的紫毛大衣铺好座位。
- 22: 他嘱我路上小心，夜里要警醒些，不要受凉。
- 23: 又嘱托茶房好好照应我。
- 24: 我心里暗笑他的迂；他们只认得钱，托他们只是白托！
- 25: 而且我这样大年纪的人，难道还不能料理自己么？我现在想想，我那时真是太聪明了。
- 26: 我说道：“爸爸，你走吧。”
- 27: ”他往车外看了看，说：“我买几个橘子去。”
- 28: 你就在此地，不要走动。
- 29: ”我看那边月台的栅栏外有几个卖东西的等着顾客。
- 30: 走到那边月台，须穿过铁道，须跳下去又爬上去。
- 31: 父亲是一个胖子，走过去自然要费事些。
- 32: 我本来要去的，他不肯，只好让他去。

2.

```
import requests
import time
import re

API_KEY = "sk-78e664727760463394135e9246255cfa" #替换成你自己的密钥
API_URL = "https://api.deepseek.com/v1/chat/completions"

headers = {
    "Content-Type": "application/json",
    "Authorization": f"Bearer {API_KEY}"
}

emotion_results = []

for num, sent in numbered_sentences:
    prompt = f"""你是一个情感分析助手。请对下面这句话做如下分析：

1. 提取或生成一个最能代表情绪的词语（只能一个）；
2. 对整句话的情绪进行打分，范围是 -1（非常负面）到 1（非常正面），中性为 0。

直接用如下格式输出：
情感词：XXX
情感值：X.X

句子：{sent}"""

    data = {
        "model": "deepseek-chat",
        "messages": [{"role": "user", "content": prompt}],
        "temperature": 0.3
    }

    try:
        response = requests.post(API_URL, headers=headers, json=data)
        time.sleep(0.1) # 控制请求频率

        if response.status_code == 200:
            content = response.json()[0]['message']['content']

            # 宽容匹配格式，如：情感词：悲伤、情感值：-0.6
            match = re.search(r"情感词[: ]?\s*([\n\r: \s]+\s*[\n\r]+情感值[: ]?\s*(-?\d+(?:\.\d+)?)", content)
            if match:
                word = match.group(1)
                score = float(match.group(2))
                emotion_results.append((num, sent, word, score))
            else:
                print(f"⚠️ 无法匹配] 第{num}句: {sent}")
                print("模型输出:", content)
                emotion_results.append((num, sent, "无", 0.0))
        else:
            print(f"❌ API错误 {response.status_code}")
            print(response.text)
            emotion_results.append((num, sent, "无", 0.0))

    except Exception as e:
```

2.

```
File Edit View Insert Cell Kernel Widgets Help Trusted Python 3
In [4]: for item in emotion_results:
        print(f"句子{item[0]}: 情感词: {item[2]}, 情感值: {item[3]}")

句子1: 情感词: 思念, 情感值: -0.3
句子2: 情感词: 悲伤, 情感值: -0.8
句子3: 情感词: 悲伤, 情感值: -0.8
句子4: 情感词: 悲伤, 情感值: -0.8
句子5: 情感词: 慰藉, 情感值: 0.3
句子6: 情感词: 沉重, 情感值: -0.8
句子7: 情感词: 悲伤, 情感值: -0.8
句子8: 情感词: 平静, 情感值: 0.0
句子9: 情感词: 平静, 情感值: 0.0
句子10: 情感词: 平静, 情感值: 0.1
句子11: 情感词: 关切, 情感值: 0.6
句子12: 情感词: 担忧, 情感值: -0.5
句子13: 情感词: 淡然, 情感值: 0.1
句子14: 情感词: 犹豫, 情感值: 0.1
句子15: 情感词: 担忧, 情感值: -0.3
句子16: 情感词: 平静, 情感值: 0.0
句子17: 情感词: 中性, 情感值: 0.0
句子18: 情感词: 无奈, 情感值: -0.3
句子19: 情感词: 忙碌, 情感值: 0.0
句子20: 情感词: 懊悔, 情感值: -0.3
句子21: 情感词: 温暖, 情感值: 0.6
句子22: 情感词: 关切, 情感值: 0.7
句子23: 情感词: 温暖, 情感值: 0.7
句子24: 情感词: 轻蔑, 情感值: -0.5
句子25: 情感词: 自嘲, 情感值: -0.3
句子26: 情感词: 不舍, 情感值: -0.3
句子27: 情感词: 温暖, 情感值: 0.6
句子28: 情感词: 关切, 情感值: 0.3
句子29: 情感词: 平淡, 情感值: 0.0
句子30: 情感词: 疲惫, 情感值: -0.3
句子31: 情感词: 无奈, 情感值: -0.3
句子32: 情感词: 无奈, 情感值: -0.3
句子33: 情感词: 感伤, 情感值: -0.3
句子34: 情感词: 艰难, 情感值: -0.3
句子35: 情感词: 努力, 情感值: 0.3
句子36: 情感词: 悲伤, 情感值: -0.8
句子37: 情感词: 克制, 情感值: -0.3
句子38: 情感词: 恐惧, 情感值: -0.8
句子39: 情感词: 欣慰, 情感值: 0.3
句子40: 情感词: 温情, 情感值: 0.3
句子41: 情感词: 关切, 情感值: 0.4
句子42: 情感词: 温暖, 情感值: 0.3
句子43: 情感词: 轻松, 情感值: 0.6
句子44: 情感词: 牵挂, 情感值: 0.3
句子45: 情感词: 失落, 情感值: -0.3
句子46: 情感词: 不舍, 情感值: -0.3
句子47: 情感词: 悲伤, 情感值: -0.8
句子48: 情感词: 低落, 情感值: -0.8
句子49: 情感词: 自豪, 情感值: 0.6
句子50: 情感词: 颓唐, 情感值: -0.8
```

2.

FileEditViewInsertCellKernelWidgetsHelp

TrustedPython 3

+

⌕

📄

⬆

⬇

▶ Run

⏮

⏪

Code

句子52: 情感词: 郁怒, 情感值: -0.8
句子53: 情感词: 失落, 情感值: -0.5
句子54: 情感词: 欣慰, 情感值: 0.6
句子55: 情感词: 悲伤, 情感值: -0.8
句子56: 情感词: 怀念, 情感值: 0.3
句子57: 情感词: 无奈, 情感值: -0.5
句子58: 情感词: 怅惘, 情感值: -0.5

▶ In [5]:

```
import matplotlib.pyplot as plt
from matplotlib import rcParams

# 设置中文字体 (适配不同系统)
# plt.rcParams['font.family'] = 'SimHei' # 黑体, 适用于 Windows
plt.rcParams['font.family'] = 'Heiti TC' # mac对应的黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号 '-' 显示为方块的问题

# 如果你使用的是 macOS, 可换为:
# plt.rcParams['font.family'] = 'Heiti TC'

# 如果是 Linux (如 Ubuntu) 且安装了中文字体:
# plt.rcParams['font.family'] = 'Noto Sans CJK SC'

# 画图
x = [item[0] for item in emotion_results]
y = [item[3] for item in emotion_results]
labels = [item[2] for item in emotion_results]

plt.figure(figsize=(12, 6))
plt.plot(x, y, linestyle='--', marker='o', color='blue')

# 添加每个点的中文情感词标签
for i in range(len(x)):
    plt.text(x[i], y[i] + 0.05, labels[i], ha='center', fontsize=9)

plt.title("散文情感词时间序列图")
plt.xlabel("句子编号")
plt.ylabel("情感值 (-1 到 1)")
plt.ylim(-1.1, 1.1)
plt.grid(True)
plt.tight_layout()
plt.show()
```

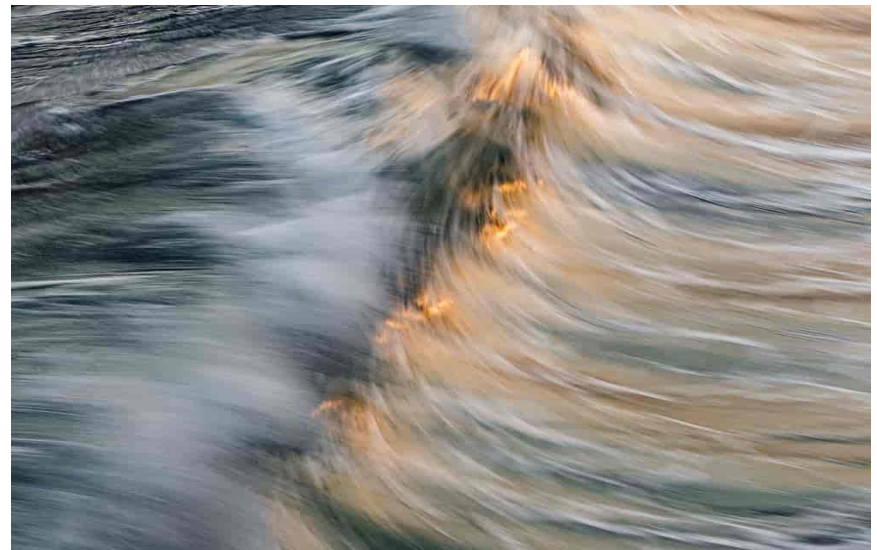
D:\Anaconda\lib\site-packages\matplotlib\font_manager.py:1331: UserWarning: findfont: Font family ['Heiti TC'] not found. Falling back to DeJaVu Sans
(prop.get_family(), self.defaultFamily[fonttext]))

<Figure size 1200x600 with 1 Axes>

▶ In []:

PART 05

知识图谱理念



医疗知识图谱的核心价值



语义化医疗知识整合

通过本体建模将分散的医疗数据（如疾病、症状、药品、指南）转化为结构化关联网络，实现跨学科知识的深度互联。例如，将"十二指肠溃疡"与消化内科概念、治疗方案、相关药物等实体建立60余种医学关系。

智能决策支持

基于图谱推理引擎，为临床诊疗提供多维度决策依据。如丁香园系统可自动关联患者症状与相似病例库，辅助医生快速定位诊疗方案，使肝癌早期诊断准确率提升至89%。

动态知识演化

通过NLP算法实时捕捉最新医学文献和临床指南更新，动态扩展图谱边界。例如自动识别新发布的药物相互作用研究，并更新至药品禁忌知识节点。

数据采集与结构化处理

01

多源异构数据融合

整合电子病历、医学文献、药品说明书等结构化与非结构化数据，采用本体映射技术解决语义冲突。浙江省医疗平台通过实体消歧算法，将237家医院差异数据统一为标准本体，清洗效率提升15倍。

02

专业医学团队校验

由临床专家团队维护核心医学概念（如ICD-10疾病分类），确保术语准确性。同时通过众包机制收集医生社区讨论内容，补充真实世界证据。

03

机器学习增强构建

应用图神经网络挖掘潜在关联，如华大基因通过基因-药物关联图谱发现新的肿瘤生物标志物组合，使靶向治疗有效率从42%提升至67%。

04

质量控制闭环

建立术语冲突检测、关系置信度评估等机制，例如用药监护系统每年自动校验超5万条药物相互作用数据，错误率低于0.2%。

实际应用场景与效果分析



精准诊疗推荐

中山大学附属医院的放疗方案推荐系统，基于肿瘤位置图谱与剂量知识库，3分钟内生成个性化计划，误差率 $<2\%$ 。临床决策响应速度提升8倍。

智能健康服务

丁香医生APP利用症状-疾病图谱实现AI分诊，覆盖1.2亿用户。通过KBQA技术解析自然语言问询，准确率较传统检索提升40%。

医药研发加速

上海药物所构建的化合物-靶点图谱结合强化学习算法，将苗头化合物发现周期从24个月压缩至6个月，已推动多个新药进入临床试验阶段。

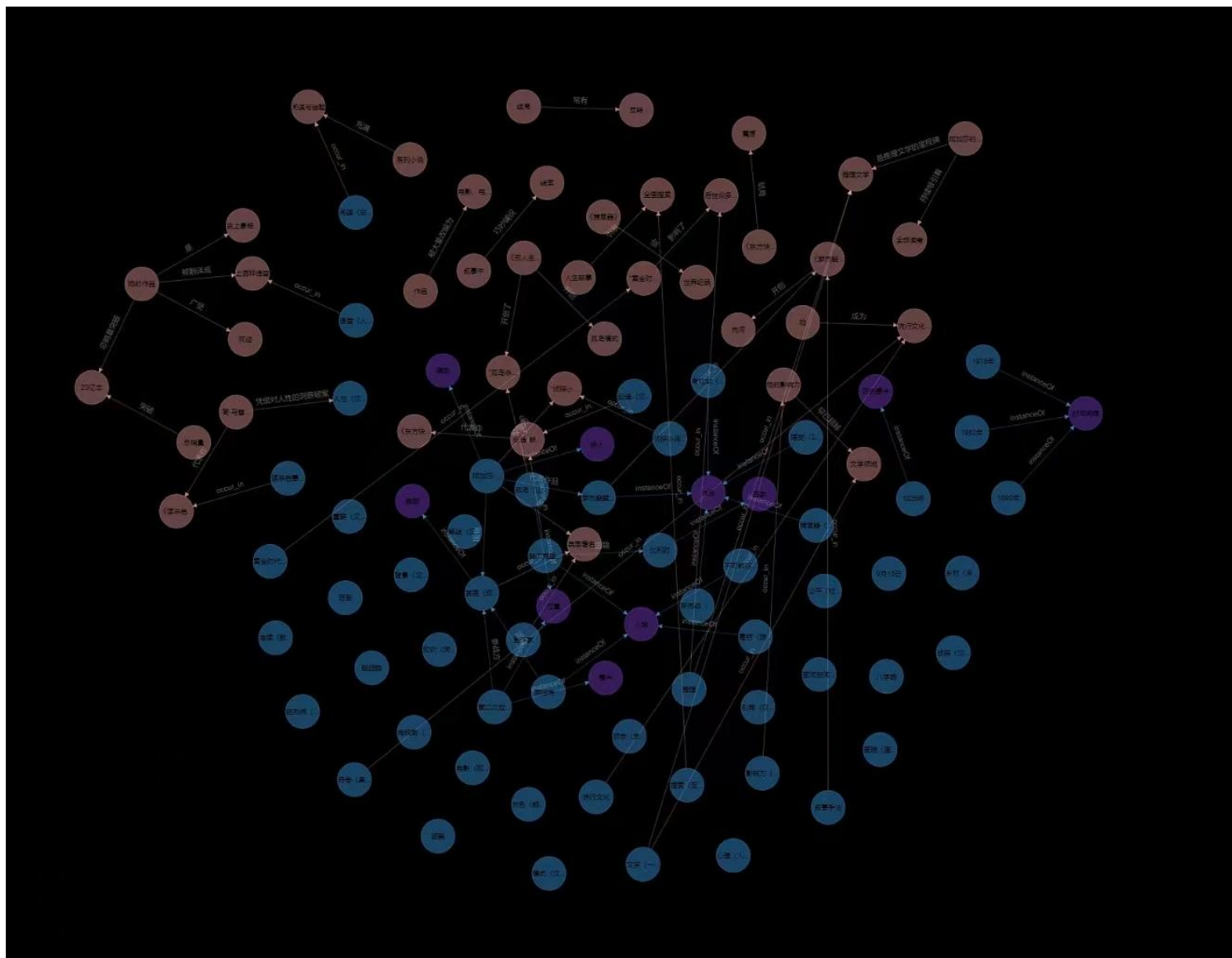


PART 06

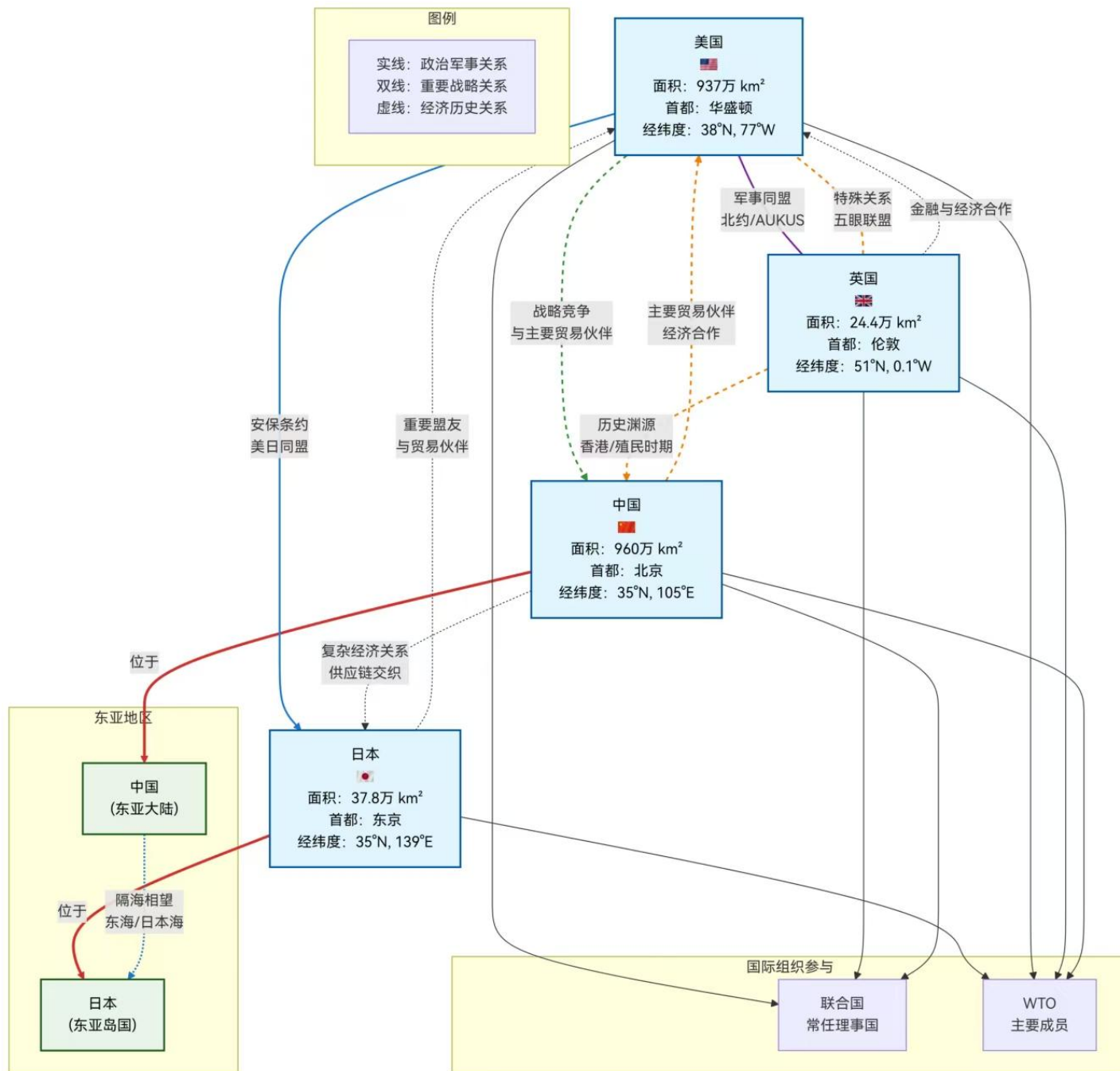
知识图谱工具



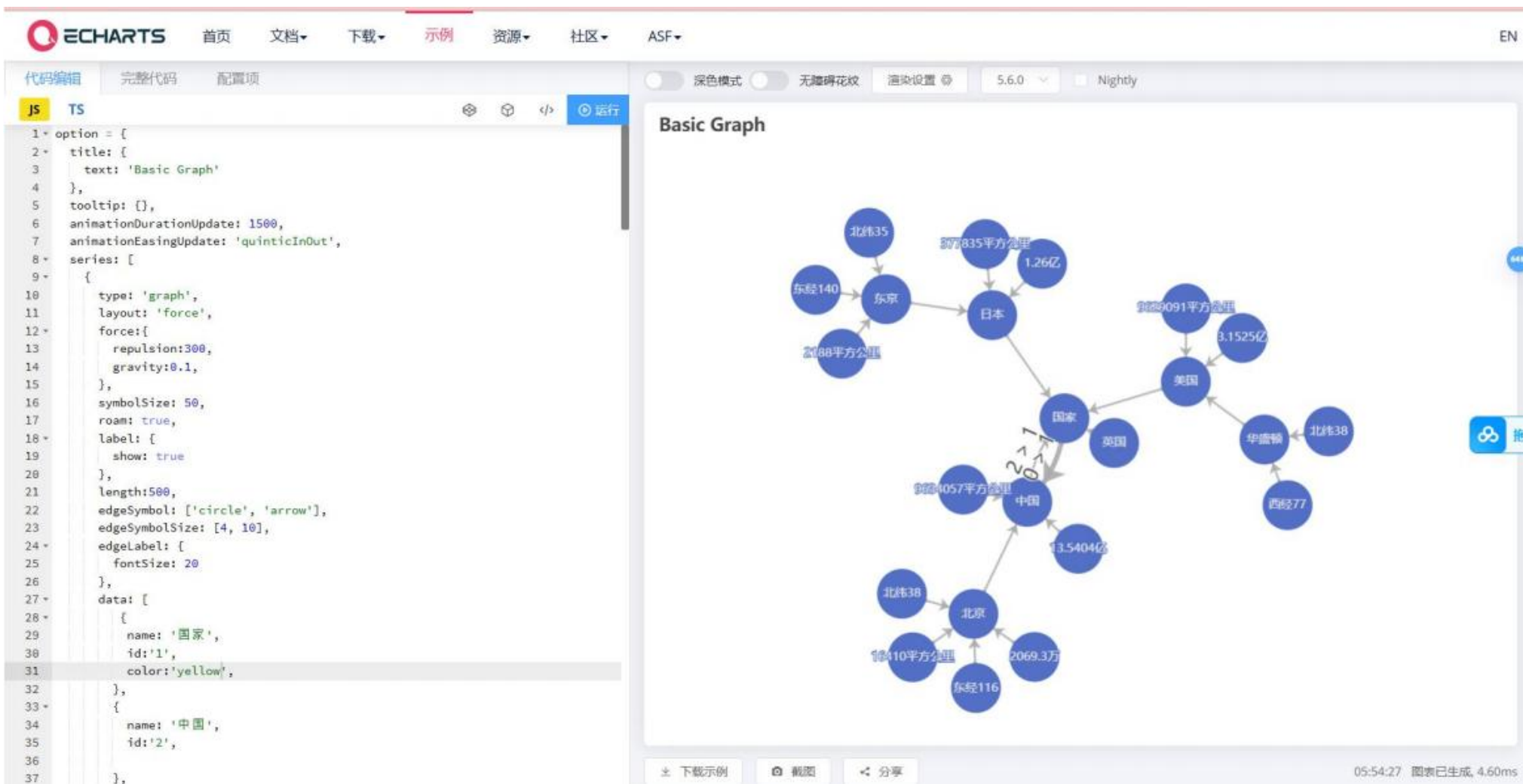
1.



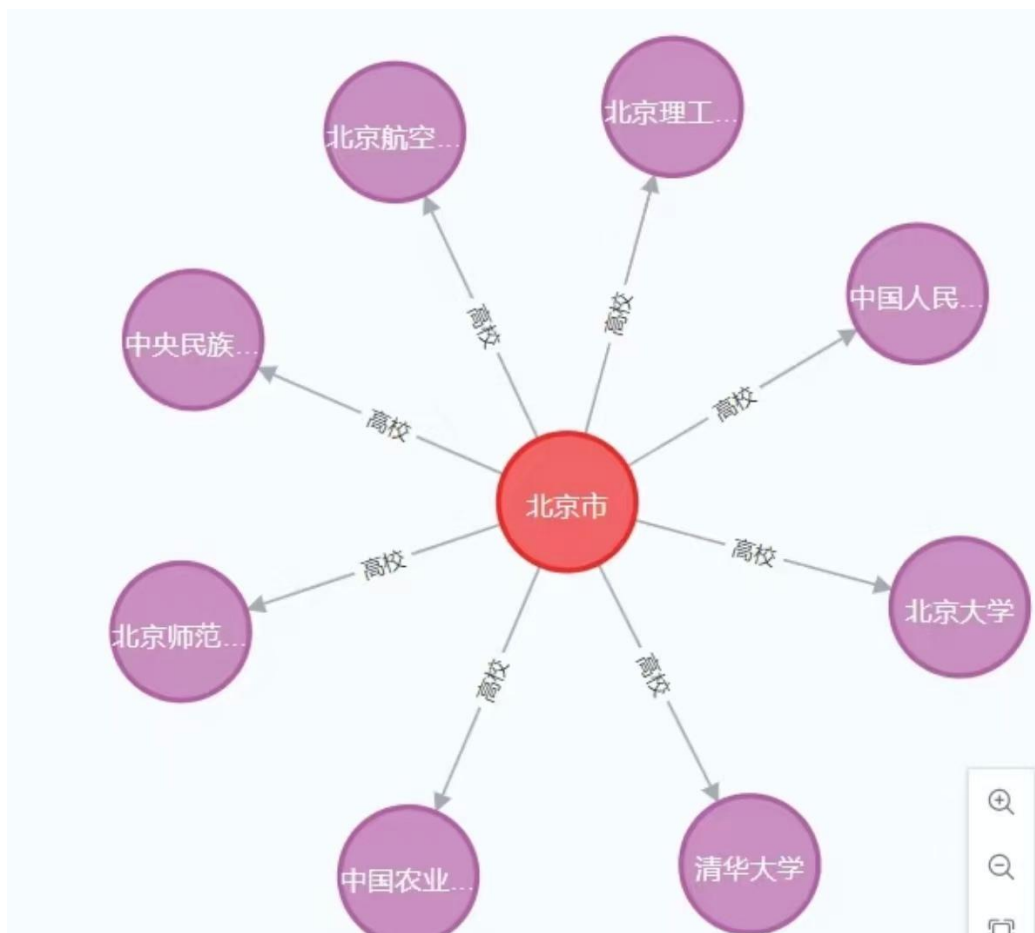
2.



3.



4.



谢谢观看

W o r k s u m m a r y r e p o r t

Type your content here, either by copying your text, or by text, Type your content here,
either by copying your text, or by copying after the text,